Coding Conventions & Standards

Table of Contents

1. Introduction

Coding conventions and standards are a set of guidelines and best practices that ensure consistency, readability, maintainability, and reliability of code across development teams and projects. Adhering to these standards is crucial for team collaboration, reduces bugs, and improves overall software quality.

2. Importance of Coding Standards

- Promotes code consistency across teams
- Facilitates easier debugging and maintenance
- Enhances code readability
- Reduces onboarding time for new developers
- Ensures better collaboration
- Minimizes common programming errors

3. General Coding Guidelines

- Use meaningful and descriptive variable names
- Write modular and reusable code
- Avoid deeply nested structures
- Use consistent indentation (usually 2 or 4 spaces)
- Limit line length to 80-120 characters
- Write self-documenting code

4. Java Coding Conventions

- Follow CamelCase naming: `ClassName`, `methodName`, `variableName`
- Constants should be in uppercase: `MAX_COUNT`
- Always use access modifiers (`private`, `protected`, `public`)
- Package naming: lowercase with dots, e.g., `com.example.utils`
- Follow JavaDoc standards for class and method documentation
- Use try-with-resources for closing resources

5. HTML Coding Conventions

- Use lowercase for tags and attributes
- Close all tags properly, including self-closing tags
- Use double quotes for attribute values
- Indent nested elements properly
- Group related elements using semantic tags (`section`, `article`, `nav`, etc.)
- Add alt attributes for images

6. JavaScript Coding Conventions

- Use CamelCase for variables and functions
- Use `const` and `let` instead of `var`
- Avoid global variables
- Use arrow functions for short anonymous functions
- Place JavaScript code in external files where possible
- Use === instead of ==
- Handle asynchronous code using Promises or async/await

7. React.js Coding Standards

- Use functional components with hooks
- Split UI into reusable components
- Name components with PascalCase
- Separate concerns: use dedicated folders for components, pages, hooks, etc.
- Prop naming should be descriptive and consistent
- Avoid inline styles; prefer CSS modules or TailwindCSS
- Use ESLint and Prettier for formatting and linting

8. SQL Coding Guidelines

- Use uppercase for SQL keywords: SELECT, INSERT, WHERE
- Use lowercase for table and column names
- Indent nested queries
- Write joins explicitly
- Avoid SELECT *; specify column names
- Use aliases for table names
- Keep statements simple and readable

9. TailwindCSS Best Practices

- Use utility classes in logical order: layout > box model > typography > state
- Extract repeated styles into components
- Use `@apply` in CSS for reusability
- Customize the config file for consistent design tokens
- Avoid long, cluttered className attributes
- Use responsive prefixes for adaptive design

10. Spring Boot Coding Standards

- Follow standard project structure: `controller`, `service`, `repository`
- Use dependency injection for managing beans
- Keep business logic in service layer
- Use DTOs to separate internal models from external responses
- Handle exceptions using `@ControllerAdvice`
- Secure endpoints with Spring Security

11. PostgreSQL Conventions

- Use snake_case for table and column names
- Primary keys: `id`, foreign keys: `user_id`, `product_id`

- Always use meaningful constraint names
- Avoid redundant indexes
- Use `TEXT` for variable-length strings and `VARCHAR(n)` only with justification
- Normalize data but avoid over-normalization

12. API Naming and Design Standards

- Use RESTful conventions: nouns for resources, verbs in HTTP methods
- URI format: `/api/v1/resource`
- Use consistent naming: `getUser`, `createOrder`
- Return proper HTTP status codes
- Document APIs using Swagger/OpenAPI
- Secure APIs with tokens (e.g., JWT)

13. Version Control Guidelines

- Use Git for source control
- Follow Git branching model (main, develop, feature, hotfix)
- Write clear, meaningful commit messages
- Commit small, incremental changes
- Review code via pull requests before merging
- Use `.gitignore` properly to exclude sensitive and unnecessary files

14. Code Commenting Best Practices

- Use comments to explain "why", not "what"
- Keep comments up-to-date
- Use block comments for large descriptions
- Use line comments for small clarifications
- Avoid redundant comments

15. Error Handling Standards

- Handle expected exceptions using try/catch
- Avoid swallowing exceptions
- Log meaningful error messages
- Return appropriate error codes and messages in APIs
- Use centralized exception handling mechanisms

16. Security and Safety Coding Standards

- Sanitize all user inputs
- Use HTTPS for secure communication
- Encrypt sensitive data at rest and in transit
- Implement role-based access control (RBAC)
- Keep dependencies updated
- Scan codebase for known vulnerabilities

17. Conclusion

Coding conventions and standards not only bring discipline to code development but also enhance teamwork and long-term maintainability. Following these best practices ensures high-quality, secure, and efficient software delivery. Developers should regularly revisit and update these standards based on evolving technologies and team requirements.